Coding Guide : TYPE-1.0

2021.11.30 update

株式会社 Penseur

目次

1. はじめに
   1.1. ガイドライン制定の目的
   1.2. 基本的な考え方
2. 基本事項
   2.1. 動作検証ブラウザ
   2.2. 注意
   2.3. 画像のファイル名・命名規則
   2.4. インデント
3. 開発環境
   3.1. 基本構造
   3.2. wordpress 想定
4. HTML / EJS
   4.1. 基本
   4.2. HEAD
   4.3. コメントアウト
   4.4. EJS(テンプレートエンジン)
5. CSS/Sass
   5.1. 基本
   5.2. 設計
   5.3. 共通事項
   5.4. 命名規則
   5.5. Foundation
   5.6. Layout
   5.7. Object
      5.7.1. component
      5.7.2. project
      5.7.3. utility
6. JS
   6.1. 基本事項
   6.2. CDN と npm について
   6.3. script タグの属性

1. はじめに

1.1. ガイドライン制定の目的

この文書(以下「ガイドライン」とします)は、株式会社 penseur が Web サイト制作を行う際の記述ルールなどを定めたものです。 本ガイドラインを定めることにより、以下の効果を期待するものとします。

- クライアントへ提供する商品品質の均一化
- 社内における更新性の向上

本ガイドラインは、新たなルールが必要な際に検討をし、随時更新を行います。

1.2. 基本的な考え方

- チーム内での一貫性の共有
- 再利用しやすい
- 試行錯誤は大歓迎

2. 基本事項

2.1. 動作検証ブラウザ

Windows

- Edge 最新版
- Firefox 最新版
- Google Chrome 最新版

Mac

- Safari 最新版
- Firefox 最新版
- Google Chrome 最新版

iPhone

- iOS 11 ~ 最新 Safari

Android

- Android 5.1 ~

2.2. 注意

IE 対応、Android 4.4 以下対応は別途費用を請求すること。
IE は、システムの仕様上必須の場合のみ対応。 以下、IE / Android 4.4 以下を別途費用とする概要です。

IE

- 公式から非推奨であると勧告されているという理由があります。
- 実際 IE を検証環境に入れることで、デバックに工数がかかるデメリットがあります。

Android 4.4 以下

- 4.4 以下の端末標準ブラウザは Chrome ではなく、端末毎にある専用ブラウザとなっており
  使用できる css にも制限がある為、検証対象外とします。

2.3. 画像のファイル名・命名規則(CSS / JS は後述)

画像

- ファイル名は、「種類 + 個性」or「個性 + 種類」を基本とします。
  (例)「bg_mv」 or 「mv_bg」
- 単語同士の組み合わせには _ を使用します。
- 数字は「01」をはじめの数値として設定し、以降「02」「03」と続きます。
- 3 桁以上の数字の仕様が想定される場合には、はじめの数値を「001」と設定します。

禁止事項

- – は使用しないこと - ダブルクリックした際、ハイフンがあると全文コピペが出来ない為です。

2.4. インデント

- インデントは半角 2 つとします。
- 各エディタで Tab の設定が可能な場合は、2と設定してください。
- 案件都合による「インデントなしでの左詰め」などのルールは、案件優先とします。

3. 開発環境

案件規模や要件によって変動しますが、src 配下の構造は原則同じとします。
`src/scss/`と`src/js/`内の構造については、「5. CSS/Sass」「6. JS」にて後述。

3.1. 基本構造

```
/
├─.gitignore
├─package-lock.json
├─package.json
├─public_html
|  ├─assets
|  |  ├─img
|  |  ├─js
|  |  └─css
|  └─index.html
├─readme.md
├─src
|  ├─ejs
|  ├─img
|  ├─js
|  └─scss
└─webpack.config.js
```

3.2. WordPress

3.2.1. 開発環境について

共有ドライブの以下にあるテンプレートを使用する。 ◆Web 事業部共通/98_フロント開発環境/front_dev_wordpress/front_dev_wordpress.zip

3.2.2. 基本構造

```
/
├─.gitignore
├─package-lock.json
├─package.json
├─public_html
|  └─wp-content
|     └─themes
|         └─thme_name
|             ├─footer.php
|             ├─front-page.php
|             ├─header.php
|             ├─index.php
|             └─style.css
|
├─readme.md
├─src
|  ├─ejs
|  ├─img
|  ├─js
|  ├─php
|  |  ├─footer.php
|  |  ├─front-page.php
|  |  ├─header.php
|  |  └─index.php
|  └─scss
└─webpack.config.js
```

3.2.3. 導入について

共有ドライブの以下にある手順書を参照。 ◆Web 事業部共通/02*システム制作共通/cms/wordpress/01*概要/rev004/

3.3.4. 使用プラグインについて

3.2.3 に記載のある導入推奨のプラグインについては、共有ドライブの以下を参照。 ◆Web 事業部共通/02*システム制作共通/cms/wordpress/03*プラグイン/ ※MW WP FORM に関しての tips が、◆Web 事業部共通/02_システム制作共通/cms/wordpress/00_tips/ にあります

4. HTML / EJS

4.1. 基本

文字コードは「UTF-8(BOM 無)」。
各要素、属性および属性値の記述には小文字を使用します。
引用符はダブルクオートを使用します。
改行コードは LF とします。
エンプティ要素(img, br, meta, link など)の閉じタグは「なし」で統一します。

同階層へのパス指定には「./」を使用します。

```
<img src="./common/img/test.png">
```

4.2. HEAD

head 要素内に含まれる要素は以下の内容、順序で記述します。

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>ページタイトル</title>
<meta name="description" content="ページ説明">
<link rel="stylesheet" href="/assets/css/style.css">
<script src="/assets/js/script.js"></script>
```

JavaScript および CSS ファイルは原則として外部ファイル化して読み込み、インラインでの使用を禁止します。
ただし、インラインで使用しなければ動作しないものに関してはこの限りではありません。

4.3. コメントアウト

コメントアウトは以下のように、記述してください。

```
<!-- 何かしらのメモ -->
<div></div>
<!-- end 何かしらのメモ -->
```

4.4. EJS(テンプレートエンジン)

テンプレートの開発環境では、ejs を採用しています。
特徴として

- 学習コストが低い
- include 機能が header/footer を読み込むのに便利
- 共通で使用できるの DOM を include で運用することにより効率 up
- if / for 文を使用できる為、出し分けやダミーの同一パターンの記述が減る
- json を併用することで、meta 情報の一括管理なども出来る

などがあります。

5. CSS/Sass

5.1. 基本

- 画面への装飾は原則的に CSS でのみ行います。
- CSS ハックの使用は最小限に抑え、バリデーションエラーとなる CSS ハックは使用しません。
- **clearfix は原則使用しません。**

5.2. 設計

一番大事な設計部分となります！
CSS 設計の基盤に FLOCSS を採用します。
命名規則などは、オリジナルから少々アレンジを加えるため
詳細は後述の「5.4. 命名規則」をご確認ください。
FLOCSS について(https://github.com/hiloki/flocss)
以下、テンプレート内の src/scss/内のディレクトリ構造となります。
ディレクトリについては、後述の 5.5. ~ 5.7.にて解説します。

```
src
└scss
    ├─foundation
    │  ├─ _base.scss
    │  ├─ _init.scss
    │  ├─ _mixin.scss
    │  ├─ _reset.scss
    │  └─ _variable.scss
    │
    ├─layout
    │  ├─ _footer.scss
    │  ├─ _header.scss
    │  └─ _init.scss
    │
    ├─object
    │  ├─ _init.scss
    │  ├─component
    │  │  └─ _init.scss
    │  ├─project
    │  │  └─ _init.scss
    │  └─utility
    │     └─ _init.scss
    │
    └─ style.scss
```

5.3. 共通事項

- style.scss で各ディレクトリの `_init.scss` を import し、style.css として出力する。
- 各ディレクトリにて、新規 scss ファイルを追加する際は、各々の階層に存在する `_init.scss` に import 文も追記してください。
- 出力ファイル以外には、`_〇〇.scss` というように _ を先頭につけてください。

5.4. 命名規則

基本的には FLOCSS の命名規則を踏襲します。
それぞれの class にはカテゴリ毎に決められた prefix をつけます。

- layout は`l-`
- project は`p-`
- component は`c-`
- utility は`u-`

以下は、命名規則の例です。

```
l-block_element--modifier
```

本来の FLOCSS(BEM 記法)では、block と element の接続は_を 2 つとしていますが
penseur では、_を 1 つとします。

element 同士の接続でも同様に、オリジナルでは_を 2 つとしていますが penseur では、

```
l-block_elementElement
```

のように、ロウアーキャメルで記述します。

modifier を使用する際は

```
<p class="c-text c-text--red"></p>
```

のように、使用してください。
class 名は明確な名称を心がけましょう。
※component では、ユニークな固有名詞は避けましょう。
　project には、使用して問題ありません。

5.5. Foundation

- デフォルトの style のリセット → `_reset.scss`
- 便利関数の設定 → `_mixin.scss`
- 各変数の管理 → `_variable.scss`
- タグ別の調整 → `_base.scss`

上記が主な役割となります。
テンプレートに初期で配置されているファイルから構成が変わることは、ほとんどないかと思います。
案件ごとに、内容を更新してください。

5.6. Layout

ページ共通で揺るがぬことがないレイアウトを管理します。
header、footer、コンテンツ幅を管理する wrap などが該当します。

5.7. Object

FLOCSS のメインコンテンツとなります。
Object 内では

- component
- project
- utility

の 3 つのディレクトリに分けて管理します。

5.7.1. component

ページで使用率の高い、共通パーツを管理します。
ファイル名の命名は、どんな役割を持っているのか明確に名付けてください。
`_btn.scss` (ボタン全般を管理する component)などなど。。

5.7.2. project

FLOCSS の概念において、イメージがしにくいカテゴリかと思います。
役割のイメージとしては、component 同士をつなぎ合わせる糊の役割となります。
TOP ページなど、ユニークな style が発生する際にも、project にて管理します。
以下は、project の使用例となっています。

```
<div class="p-indexNews">
  <ul class="c-news">
    <li class="c-news_list">
      <a href="" class="c-news_link">hogehoge</a>
    </li>
  </ul>
  <div class="p-indexNews_btn">
    <a href="" class="c-btn">ボタンどす</a>
  </div>
</div>
```

上記の`p-indexNews`は、TOP ページの News エリアの枠組みとして
上下などに margin を設けたり、幅指定などを行う想定の project です。
`c-btn`を`p-indexNews_btn`で wrap しているのは
使用回数の多い`component`に固有の width や margin を設定すると
汎用性が失われるため、`p-indexNews_btn`で枠組みを構成しています。

5.7.3. utility

局所的に使うような class を管理します。

- ここのテキストだけ`text-align`つけたい... → `u-ta`
- 上にちょっとだけ`margin`つけたいけど、そのために project 増やすのもなあ... → `u-mt30`

上記のような形で、使用してください。
便利な class たちではありますが、使用過多は**汎用性を失う原因**となるので
なるべく設計力で乗り切りましょう。

6. JavaScript

6.1. 基本事項

- 基本的に ES6 で書くこと
  初めて js 書きます！という方は jQuery ウェルカムです！！
- シングルクォートを使用すること
- 実際の処理の記述まで縛ることは避けたいので、基本的には自由です。

6.2. CDN と npm について

- CDN 使用時は、url が変更した際のリスクを承知の上で使用すること
- npm install でプラグイン等を install するのは各案件ごとに自由に行ってください

6.3. script タグの属性

script タグには、`async` / `defer`属性をつけることができ、それぞれ用途が異なります。
基本的には`defer`を設定してください。
defer を設定することで、body 前後で script は読み込まずとも
head 内に、まとめて script を記述することができます。
@phanectさんの Qiita 記事にて解説されています。
記事 url: https://qiita.com/phanect/items/82c85ea4b8f9c373d684

6.4 ディレクトリについて

ディレクトリは以下のようになります。

```
├─ src
│  └─ modules
│      └─ ○○.js
│
└─ script.js
```

6.5 命名規則

エントリーポイントは小文字で単語 1 つのみ。
modules 内のファイル名に関しては

```
toggleClass.js
```

のように、単語を複数組み合わせる場合は`ロウアーキャメル`とします。

6.6 運用方法

前述のディレクトリ構造にある`script.js`をエントリーポイントとして、`modules`内の js をまとめて 1 ファイルとして出力します。
共通ファイルとして使用する js と、一部のページでのみ使う js を分けて、新たに js ファイルを出力したい場合は、`webpack.config.js`のエントリーポイントの設定に、以下のように追記してください。

```
entry: {
  script: `./src/js/index.js`,
  追加したいファイル名A: `./src/js/追加したいファイル名A.js`,
  追加したいファイル名B: `./src/js/追加したいファイル名B.js`,
}
```